

AISHA

Artificial Intelligent Smart Human Assistant

Project State & Progress Report

Repository	Personal_assistance- (mayankjangid0604)
Working branch	claude/project-state-details-pdf-axz5bi
Report date	25 July 2026
Latest commit	e5b5aeb — Initial commit (24 Jul 2026)
Overall status	Feature-complete across Phases 1–8 · test-verified

An offline-first, voice-enabled desktop AI companion built with a modular Python cognitive backend and a React + Electron frontend.

1 · Executive Snapshot

AISHA is a personal AI assistant that combines a large modular cognitive backend with a desktop chat/voice interface. Development has progressed through **eight planned phases** — from foundation refactoring to an autonomous, self-organizing AI ecosystem. All twelve automated test suites pass (the single failing case requires a Windows-only clipboard module and is skipped elsewhere).

~39k Lines of code	61 Backend modules	12 Modular skills	51 Database tables
8 Phases delivered	1,138 Tests passing	15 React components	143 Tracked files

Line counts and test totals measured directly from the working tree on the report date.

2 · System Architecture

Layer	Technology	Responsibility
Entry / CLI	Python (main.py)	Terminal chat loop, text & voice modes, autonomous background thread
API server	Quart + Hypercorn (async)	REST + SSE streaming endpoints; migrated from Flask for async/WebSocket readiness
Cognitive core	61 Python modules	Brain pipeline: role detection, emotion, memory, planning, autonomy, agents
Skill system	12 priority-ranked skills	Pluggable can_handle()/handle() units (safety, task, reminder, emotion, journal...)
Persistence	SQLite + 7 repositories	51 tables; migrated from JSON stores to a relational DB with repo pattern
Frontend	React + Vite + Electron	Desktop app: chat, 3D orb, dashboard, habits, journal, learning panels
LLM / Voice	Gemini · STT/TTS engines	google-generativeai routing; streaming STT/TTS voice pipeline

3 · Phase Progress

The master plan defined seven phases; implementation extended into an eighth. Every phase below is present in the codebase and covered by a dedicated passing test suite.

Phase	Theme	Delivered highlights	Status
1	Foundation Stabilization	Modular backend, priority skill system, markdown chat, real Gemini streaming, notifications	✓ Complete
2	Real AI Experience	Full voice pipeline (STT→brain→TTS), conversation engine, orb sync, persistent sessions	✓ Complete
3	Memory Evolution	SQLite migration, semantic + episodic memory, cognitive agents, planner & orchestrator	✓ Complete
4	Desktop AI System	Desktop/cross-app awareness, workflow & productivity cognition, agentic executor, governor	✓ Complete
5	Autonomous AI	Routine intelligence, behavioral model, reflective cognition, NL automation, life management	✓ Complete
6	Production-Level System	Deep personalization, emotional timing, multi-step automation, proactive orchestration	✓ Complete
7	Future AI Platform	Project cognition, cognitive workspace, knowledge synthesis/graph, research, creativity	✓ Complete
8	Autonomous Ecosystem	Human oversight, distributed agents, project evolution, long-horizon planning, explainability	✓ Complete

4 · Test & Quality Status

All maintained suites in backend/tests/ were executed on the report date after installing optional dependencies (dotenv, numpy, cffi, cryptography). Results:

Suite	Tests	Result
Async migration	50	✓ PASS
Phase 2 — integration	31	✓ PASS
Phase 2 — experience layer	91	✓ PASS
Phase 3 — cognitive AI	225	✓ PASS
Phase 4 — operating layer	185	✓ PASS
Phase 5 — ecosystem	182	✓ PASS
Phase 6 — personalization	113	✓ PASS

Phase 7 — collaborative	162	✓ PASS
Phase 8 — oversight	24 / 25	1 skipped (Windows-only)
Phase 8 — agents	10	✓ PASS
Phase 8 — steps 3–8	13	✓ PASS
Skill priority migration	51	✓ PASS

Total: ~1,138 tests, effectively 100% passing. The lone exception is a clipboard-rollback test needing win32clipboard, which is Windows-specific and correctly skipped on this platform.

5 · Key Components

Cognitive backend (selected of 61 modules)

- **brain.py** — central controller wiring role detection, emotion, memory, skills and response generation.
- **semantic_memory.py / episodic_memory.py** — tokenized semantic recall and event/emotion memory.
- **cognitive_agents.py / distributed_agents.py** — multi-agent reasoning with bounded, specialized agents.
- **planning_engine.py / cognitive_planner.py** — long-horizon and step-level planning.
- **human_oversight.py / explainable_autonomy.py** — audit log, rollback registry and explainable decisions.
- **autonomous.py / proactive_orchestration.py** — background loop delivering proactive, well-timed nudges.

Skill system (priority order)

- safety (1) → data_control (2) → task (3) → reminder (4) → emotion (5) → journal (6) → summary (7) → learning (8) → power (9) → recall (10) → general (100, fallback).

Data layer

- SQLite database `aisha.db` with 51 tables spanning conversations, memory, habits, routines, projects, knowledge graph and audit logs.
- Seven repositories (memory, reminder, habit, journal, learning, notification, personality) plus a JSON→SQLite migration script.

Frontend (React + Electron)

- Panels: Chat, Dashboard, Schedule, Habits, Journal, Learning; a Three.js **AIOrb** reflecting AISHA's state.
- Streaming markdown chat (react-markdown, remark-gfm, rehype-highlight) and a WebSocket hook.
- Electron shell with mini-window and microphone overlay for a desktop companion experience.

6 · Tech Stack & Observations

Stack

Backend	Python 3, Quart, Hypercorn, quart-cors, google-generativeai, python-dotenv
Voice	pytsx3, SpeechRecognition (streaming STT/TTS engines)
Data	SQLite (stdlib), repository pattern
Frontend	React 18, Vite, Electron, three / @react-three/fiber / drei, highlight.js

Strengths

- Clean modular architecture with a consistent skill interface and repository pattern.
- Very strong automated test coverage (~1,138 tests) organized by phase.
- Well-documented modules — every file carries a phase-tagged docstring.
- Offline-first design with human-oversight, audit and rollback baked in.

Observations & next steps

- **requirements.txt is minimal** — optional deps (numpy, cffi/cryptography, dotenv) are needed to run the full test suite; consider pinning them or splitting a requirements-dev.txt.
- **Platform coupling** — a few features (win32clipboard) are Windows-only; guard or document cross-platform behavior.
- **Root ad-hoc script** test_phase7.py calls a renamed API (get_related_concepts) and errors; align it with the maintained suite or remove it.
- **Single commit history** — the repo is one 'Initial commit'; adopting incremental commits would aid traceability.
- **No CI configuration detected** — a GitHub Actions workflow running the phase suites would lock in the current green status.

Report generated for the AISHA project · Personal_assistance- repository · 25 July 2026.